Page No: 1
Date:

Name : UDHAYARAJAN J.
class : MCA - I
Subject : Data Structure using Java.
Date : 25-02-2026.
Roll no : 9202561
Type : CCE - 2.

1) write a java program for to sort given array by using following sorting and searching techinque.

a) Bubble sort :

Code :

```
public class Bubblesort {
    public static void main(String[] args) {
        int[] arr = {45, 23, 87, 12, 3, 89}; // Given array
        int n = arr.length; // 6.
        int temp; // 0

        System.out.println("Before using bubble sort");
        for (int i=0; i<n; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();
        for(int i=0; i<n-1; i++) {
            for(int j=0; j<n-i-1; j++) {
                if (arr[j] > arr[j+1]) {
                    temp = arr[j];
                    arr[j] = arr[j+1];
                    arr[j+1] = temp;
                }
            }
        }
        System.out.println("after using bubble sort:");
        for(int i=0; i<n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
```

```
    }
}
```

output:

Before using bubble sort:
45 23 87 12 3 89.
After using bubble sort:
3 12 23 45 87 89

b) Selection Sort.

code:

```
public class SelectionSort {
    public static void main(String[] args) {
        int[] arr = {45, 23, 87, 12, 3, 89};
        int n = arr.length;
        int temp;
        int minIndex;

        System.out.println("Before using Selection Sort");
        for(int i=0; i<n; i++){
            System.out.print(arr[i] + " ");
        }
        System.out.println();
        for(int i=0; i<n-1; i++){
            minIndex = i;
            for(int j=i+1, j<n; j++){
                if(arr[j]<arr[minIndex]){
                    minIndex = j;
                }
            }
        }
        temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}
```

Core logic

```
        System.out.println("After using Selection Sort");
        for(int i=0; i<n; i++){
            System.out.print(arr[i] + " ");
        }
    }
}
```

Output:

Before using selection sort:

45 23 87 12 3 89

After using Selection Sort.

3 12 23 45 87 89

c) Insertion Sort:

code:

```
public class InsertionSort {
    public static void main(String[] args){
        int[] arr = {45, 23, 87, 12, 9, 89};
        int n = arr.length;

        System.out.println("Before using insertion
                            sort");
        for(int i=0; i<n; i++){
            System.out.print(arr[i] + " ");
        }
        System.out.println();
        for (int i=1; i<n; i++) {
            int key = arr[i];
            int j = i-1;
            while(j>=0 && arr[j] > key) {
                arr[j+1] = arr[j];
                j--;
            }
            arr[j+1] = key;
        }
```

```
        System.out.println("After using insertion
                            sort:");
        for(int i=0;i<n;i++){
            System.out.print(arr[i]+" ");
        }
    }
}
```

output:

Before using insertion sort:
45 23 87 12 3 89
After using insertion sort:
3 12 23 45 87 89

d) Quick sort:

code:

```
public class Quicksort {
    public static void main(String[] args) {
        int[]arr = {45, 23, 87, 12, 3, 89};
        int n = arr.length;
        System.out.println("Before using Quick sort");
        for(int i=0; i<n; i++){
            System.out.print(arr[i]+" ");
        }
        System.out.println();
        QuickSortFn(arr, 0, n-1);
        System.out.println("After using Quicksort");
        for(int i=0; i<n; i++){
            System.out.print(arr[i]+" ");
        }
    }
    static void QuicksortFn(int[] arr, int low, int high){
        if(low < high){
            int partitionIndex = partitionFn(arr,low,high);
            QuicksortFn(arr, low, partionIndex-1);
            QuicsortFn(arr, partionIndex+1, high);
        }
    }
```

```
static int partitionFn(int[] arr, int low, int high){
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i+1];
    arr[i+1] = arr[high];
    arr[high] = temp;
    return i+1;
  }
}
```

Output:

Before using Quick sort:
45 23 87 12 3 89.
After using Quick sort:
3 12 23 45 87 89.

e) Linear Search with Example and dry run. (Take input from user).

Code:

```
import java.util.Scanner;
public class LinearSearch {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter number of element");
        int n = sc.nextInt();
        int arr[]
        int[] arr = new int[n];
```

```
        System.out.println("Enter the elements");
        for(int i=0;i<n;i++){
            arr[i] = sc.nextInt();
        }

        System.out.print("Enter element to search");
        int key = sc.nextInt();

        boolean found = false;
        for(int i=0;i<n;i++){
            if(arr[i]==key){
                System.out.println("Element found
                        at position" +(i+1));
                found = true;
                break;
            }
        }

        if(!found){
            System.out.println("Element not Found");
        }

        sc.close();
    }
}
```

Take input from user

Output:

```
Enter number of elements: 5
Enter Elements:
45
56
4
8
9
Enter element to search: 9
Element found at position: 5
```

Example of output sample

Page No:
Date:

Dry Run:

```
Scanner sc = new Scanner(System.in);

System.out.println(Enter the(n) elements);
int n = sc.nextInt();

System.out.print(Enter the elements);

for (int i=0; i<n; i++) {
    arr[i] = sc.nextInt();
}

for (int i=1; i<n; i++) {
    arr[i] = sc.nextInt();
}

for (int i=2; i<n; i++) {
    arr[i] = sc.nextInt();
}

for (int i=3; i<n; i++) {
    arr[i] = sc.nextInt();
}

for (int i=4; i<n; i++) {
    arr[i] = sc.nextInt();
}

for (int i=5; i<n; i++) {
    .....
    .....
```

Enter the (n) elements
n = 5 ;

Enter the elements.

① i=0, 0<5 ; 0++
arr[0] = 45
i = 1

② i=1 ; 1<5 ; 1++
arr[1] = 56.
i = 2

③ i=2 ; 2<5 ; 2++
arr[2] = 4
i = 3

④ i=3 ; 3<5 ; 3++
arr[3] = 8
i=4.

⑤ i=4 ; 4<5 ; 4++
arr[4] = 9
i = 5

⑥ i=5 ; 5<5 ; 5++

0	1	2	3	4
45	56	4	8	9

System.out.print("enter element to search");	Enter element to search
int key = sc.nextInt();	key = 56.
boolean found = false;	found = false;
for (int i=0, i<n; i++) {(✓)(1)	i=0; 0<5; 0++ (✓).
if (arr[i] == key) {(X).	45 == 56 (X).
.....	i=1.
.....	
.....	
}	
}	
for (int i=0; i<5; i++){(✓)(2)	i=1; 1<5; 1++ (✓)
if (arr[i] == key) {(✓).	56 == 56 (✓).
System.out.println(" element at position"+(i+1));	element at position: 2
found = true;	found = true.
break;	
}	
}	
if (!found) { (X).	found = true not a.
.....	false.
}	
sc.close();	

1) Binary search with example and dry run. (Take input from user).

Code:

```
import java.util.Scanner;
import java.util.Arrays;
public class BinarySearch {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements");
        int n = sc.nextInt();
        int arr[] = new int[n];
        System.out.println("Enter elements");
        for(int i=0; i<n; i++) {
            arr[i] = sc.nextInt();
        }
        Arrays.sort(arr);
        System.out.print("Enter element to search");
        int key = sc.nextInt();

        int low = 0;
        int high = n-1;
        int mid;
        boolean found = false;
        while (low <= high) {
            mid = (low + high) / 2;
            if(arr[mid] == key) {
                System.out.println("Element found at :" +
                                   (mid+1));
                found = true;
                break;
            }
            else if(arr[mid] < key) {
                low = mid+1;
            }
            else {
```

Take input from user

Logic of Binary search

```
        }
        if (!found) {
            System.out.println("Element not found");
        }
        sc.close();
    }
}
```

output:

```
Enter number of elements: 4
Enter elements:
4
9
2
6
After sorting the elements:
2
4
6
9
Enter element in to search: 4
Element found at position: 2
```

Example of sample output

Dry run:

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter number of elements");
```

Enter number of elements

```
int n = sc.nextInt();
```

n = 4.

```
System.out.println("Enter elements");
```

Enter elements

```
for(int i=0; i<n; i++) {   (✓).
    arr[i] = sc.nextInt();   (1)
}
```

i=0 ; 0<4 ; 0++
arr[0] = 4.
i=1

```
for(int i=1; i<n; i++) {   (✓).
    arr[i] = sc.nextInt();   (2)
}
```

i=1 ; 1<4 ; 1++
arr[1] = 9.
i=2.

```
for(int i=2 ; i<n ; i++){ (✓)
    arr[i] = sc.nextInt(); (4)
}

for(int i=3 ; i<n ; i++){ (✓)
    arr[i] = sc.nextInt(); (4)
}

for(int i=4 ; i<n ; i++){ (X)
    .....
    ..... (6)
```

```
i=2 ; 2<4 ; 2++
arr[2] = 2.
i=3.

i=3 ; 3<4 ; 3++
arr[3] = 6.
i=4.

i=4 ; 4<4 ; 4++ (X).
.
```

0	1	2	3
4	9	2	6

```
Arrays.sort(arr);
```

0	1	2	3
2	4	6	9

```
System.out.println("Enter element to search");
int key = sc.nextInt();

while(low<=high){ (✓).
    mid = (low+high)/2;
    if(arr[mid] == key){ (✓).
        .....
        break;
    }
    else if(arr[mid]<key){ (✓).
        low = mid+1;
    }
    else{ (✓).
        high = mid-1;
    }
}
```

enter element to search

key = 4.

low = 0.
high = 3.
mid = (3+0)/2 => 1
if(arr[1] == 4).
4 == 4.
found = true.

Page No:
Date:

if (!found) ✓ (x).

found = true.

}

sc.close();

2) Sort following array using Bubble sort, Selection sort, Insertion sort, Quick sort and show detailed day run.

a) 45, 23, 87, 12, 3, 89.

i). Bubble sort.

[45, 23, 87, 12, 3, 89].

0	1	2	3	4	5
45	23	87	12	3	89

size(n) = 6.

initialize.
arr = [45, 23, 87, 12, 3, 89].
n = 6.

Sort:

① for(int i=0; i<n-1; i++) ✓ ① i=0; 0<(6-1); 0++

for(int j=0; j<n-i-1; j++) ✓ j=0; j<(6-0-1); 0++

② if (arr[j] > arr[j+1]) ✓ 45 > 23 (✓).

temp = arr[j];

arr[j] = arr[j+1];

arr[j+1] = temp;

}

} }

swap:

0 ⇄ 1

0	1	2	3	4	5
23	45	87	12	3	89

j=1.

②

for (int j=1 ; j<5 ; j++) {
    if (arr[j] > arr[j+1])
        swap operation ....
}

(2) inner loop

j=1 ; 1<5 ; 1++
45 > 87 (X).

0	1	2	3	4	5
23	45	87	12	3	89

j=2

for (int j=2 ; j<5 ; j++) {
    ⇒ check current and next
    ⇒ swap.
}

(3) inner loop

j=2 ; 2<5 ; 2++
87 > 12 (✓)
swap : 2 ↔ 3

0	1	2	3	4	5
23	45	12	87	3	89

j=3.

for (j=3 ; j<5 ; j++) {
    ⇒ check current and next.
    ⇒ swap. if true.
}

(4) inner loop

j=3 ; 3<5 ; 3++
87 > 3 (✓).
swap : 3 ↔ 4

0	1	2	3	4	5
23	45	12	3	87	89

j=4.

for (j=4 ; j<5 ; j++) {
    ⇒ check current and next.
    ⇒ swap if true.
}

(5) inner loop

j=4 ; 4<5 ; 4++
87 > 89 (X).

i=1 ⇐

(2) for (int i=1 ; i<5 ; i++) {
    for (int j=0 ; j<n-i-1 ; j++) {
        ⇒ check current and next.
        ⇒ swap.
    }
}

(1) inner loop

i=1 ; 1<5 ; 1++
j=0 ; 0<(5-1-1) ; 0++ (4).
23 > 45 (X).
(0) (1).

j=1.

for (int j=1 ; j<4 ; j++) {
    ⇒ check current and next
    ⇒ swap.
}

(2) inner loop

45 > 12 (✓).
swap : 1 ↔ 2

0	1	2	3	4	5
23	12	45	3	87	89

Page No:
Date:

② inner loop
```
for(int j=2 ; 2<4 ; j++){
    .....
    .....
```

j=2.

45 > 3 (✓).

swap: 2 ↔ 3

0	1	2	3	4	5
23	12	3	45	87	89

④ inner loop
```
for(int j=3; 3<4 ; j++){
    .....
    .....
```

j=3

45 > 87 (x).

j=4.

③
```
for(int i=2 ; 2<(n-1) ; i++){
```

i=2 // i=2 ; 2<5 ; 2++

① inner loop
```
    for(int j=0 ; j<n-i-1 ; j++){
        .....
        .....
```

j=0 ; 0<(6-2-1) ; 0++

23 > 12 (✓).

swap: 0 ↔ 1

0	1	2	3	4	5
12	23	3	45	87	89

⑤ inner loop
```
for(int j=1 ; 1<3 ; j++){
    .....
    .....
```

j=1.

j=1 ; 1<3 ; 1++

23 > 3 (✓).

swap: 1 ↔ 2

0	1	2	3	4	5
12	3	23	45	87	89.

③ inner loop
```
for(int j=2 ; 2<3 ; j++){
    .....
    .....
```

j=2.

j=2 ; 2<3 ; 2++

23 > 45 (x).

j=3 ; i=3.

④
```
for(int i=3 ; 3<5 ; i++){
```

i=3 ; 3<5 ; 3++

① inner loop
```
    for(int j=0 ; j<(n-i-1) ; j++){
        .....
        .....
```

j=0 ; j<(6-3-1) ; 0++

12 > 3 (✓)

swap: 0 ↔ 1

Page No:
Date:

final output ✓

0	1	2	3	4	5
3	12	23	45	87	89

② inner loop

```
for(int j=1; j<2; j++){
    .....
}
```

j = 1.
1 = 1 ; 1 < 2 ; 2++
12 > 23 (X).
j = 2.

⑥
```
for(int i=4; 4<5; i++){
    for(int j=0; j<(n-i-1); j++){
        .....
    }
```

i = 4.
i = 4 ; 4 < 5 ; 4++
j = 0 ; 0 < (6 - 4 - 1) ; 0++
3 > 12 (X).
j = 1.
i = 5

① inner loop

ii) Selection sort (Descending).

0	1	2	3	4	5
45	23	87	12	3	89.

①
```
for(int i=0; i<n-1; i++){
    midIndex = i;
```

① inner loop:
```
    for(int j=i+1; j<n; j++){
        if(arr[j] > arr[midIndex])
            midIndex = j;
    }
```

int i=0 ; 0 < 5 ; 0++
midIndex = 0.
j = 0+1 ; 1 < 6 ; 1++
23 > 45 (X).
j = 2.

swap
```
    temp = arr[midIndex];
    arr[midIndex] = arr[i];
    arr[i] = temp;
}
```

② inner loop:
```
for(int j=2; 2<6; j++){
    .....
}
```

j = 2 ; 2 < 6 ; 2++
87 > 45 (✓).
midIndex = 2.
j = 3.

③ inner loop
```
for(int j=3; 3<6; j++){
    .....
}
```

j = 3 ; 3 < 6 ; 3++
12 > 87 (X).
j = 4.

Page No:
Date:

④ inner loop: `for(int j=4; 4<6; j++){`

j=4 ; 4<6 ; 4++
3 > 87 (x).
j=5

⑤ inner loop: `for(int j=5; 5<6; j++){`

j=5 ; 5<6 ; 5++
89 > 87 (✓).
minIndex = 5
j=6.

swap operation:

i=0 ; midIndex=5
0 ⇄ 5

0	1	2	3	4	5
89	23	87	12	3.	46

i=1

② `for(int i=1; i<n-1; i++){`
`midIndex = i`

i=1 ; 1<5 ; 1++
midIndex = 1.

③ inner loop: `for(int j=(i+1); j<n; j++){`
`if(arr[j] > arr[midIndex]){`
`midIndex = j`

j=2 ; j<6 ; j++
87 > 23 (✓)
midIndex = 2.
j=3.

swap

② inner loop: `for(int j=3; 3<6; j++){`

j=3 ; 3<6 ; 3++
12 > 87 (x).
j=4.

③ inner loop: `for(int j=4; 4<6; j++){`

j=4 ; 4<6 ; 4++
3 > 87 (x).
j=5.

⑪ inner loop: `for(int j=5; 5<6; j++){`

j=5 ; 5<6 ; 5++
45 > 87 (x)
j=6.
i=1 ; midIndex=2.

swap operation:

1 ⇄ 2

0	1	2	3	4	5
89	87	23	12	3	45

i = 2.

① for(int i=2, 2<5; 2++) {
midIndex = i;
② for(int j=2+1; 3<6; 3++) {
... }

i = 2 ; 2<5 ; 2++
midIndex = 2.
j = 3 ; 3<6 ; 3++
12 > 23 (x).
j = 4.

② for(int j=4, 4<6; 3++) {
... }

j = 4 ; 4<6 ; 4++
3 > 23 (x).
j = 5

② for(int j=5; 5<6; 5++) {
...

j = 5 ; 5<6 ; 5++
45 > 23 (✓)
midIndex = 5
j = 6.
i = 2 ; midIndex = 5

swap operation:

2 ↔ 5

0	1	2	3	4	5
89	87	45	12	3	23.

i = 3.

③ for(int i=3; 3<5; 3++) {
midIndex = 3;
④ for(int j=4; 4<6; 4++) {
...

i = 3 ; 3<5 ; 3++
midIndex = 3
j = 4 ; 4<6 ; 4++
23 > 12. (✓).
midIndex = 4.
j = 5.

⑤ for(int j=5; 5<6; 5++) {
...

j = 5 ; 5<6 ; 5++
23 > 3. (✓).
midIndex = 5
j = 6.
i = 3 ; midIndex = 5

swap operation:

3 ↔ 5

0	1	2	3	4	5
89	87	45	23	3	12

(A) for(int i=4; i<5; i++){
    midIndex = 4.
    ① for(int j=5; 5<6; j++){
        .....
    }
    swap operation:

i = 4.
i=4; 4<5; i++
midIndex = 4.
j=5; 5<6; 5++
12 > 3 (✓)
midIndex = 5.
j=6.
i=4; midIndex = 5
4 ↔ 5

0	1	2	3	4	5
89	87	45	23	12	3.

final output

iii) Insertion Sort.

0	1	2	3	4	5
45	23	87	12	3	89

i = 5

① for(int i=1; i<n; i++){
    int key = arr[i];
    int j = i-1;
    ② while(j>=0 && arr[j]>key){
        arr[j+1] = arr[j];
        j--;
    }
    arr[j+1] = key;
}

i=1; 1<6; i++
key = 23
j = 1-1 => 0.
0>=0 && 45 > 23 (✓)
arr[1] = arr[0];
j = -1.
arr[-1+1] = 23.

0	1	2	3	4	5
23	45	87	12	3	89.

i = 2.

② for(int i=2; i<6; i++){
    int key = arr[i];
    int j = i-1;
    ③ while(j>=0 && 45 > 87){
    }
    arr[j+1] = key;

i=2; 2<6; i++.
key = 87.
j = 2-1 => 1.
1>=0 && 45 > 87 (X)
arr[1+1] = 87.

0	1	2	3	4	5
23	45	87	12	3	89

i = 3.

③ for(int i=3; 3<6; 3++){
  int key = arr[3];
  int j = 3-1;
  while(2>=0 && 87>12){
    arr[2+1] = arr[2];
    2--;
  }

i=3; 3<6; 3++
key = 12.
j = 3-1 ⇒ 2.
2>=0 && 87>12 (✓)
arr[3] = arr[2]
arr[3] = 87.
2-- ⇒ 1.

① inner loop

  while(1>=0 && 45>12){
    arr[1+1] = arr[1];
    1--;
  }

1>=0 && 45>12 (✓)
arr[2] = 45
1-- ⇒ 0.

② inner loop

  while(0>=0 && 23>12){
    arr[0+1] = arr[0];
    0--;
  }

0>=0 && 23>12 (✓)
arr[1] = 23.
0-- ⇒ -1.

③ inner loop

  arr[-1+1] = key;

arr[0] = 12.

0	1	2	3	4	5
12	23	45	87	3	89

i=4

④ for(int i=4; 4<6; 4++){
  int key = arr[4];
  int j = 4-1;
  while(3>=0 && 87>3){
    arr[3+1] = arr[3]
    3--;
  }

i=4; 4<6; 4++
key = 3.
j = 4-1 ⇒ 3.
3>=0 && 87>3 (✓).
arr[3+1] = 87.
3-- ⇒ 2.

① inner loop

  while(2>=0 && 45>3){
    arr[2+1] = arr[2]
    2--;
  }

2>=0 && 45>3 (✓)
arr[2+1] = 45
2-- ⇒ 1:

② inner loop

```
while (i>=0 && 23 > 3) {
    arr[i+1] = arr[i]
    i--;
}
```

i>=0 && 23 > 3 (✓)
arr[2] = 23.
i-- ⇒ 0

② inner loop

```
while (0>=0 && 12 > 3) {
    arr[0+1] = arr[0];
    0--
}
```

0>=0 && 12 > 3 (✓)
arr[1] = 12
0-- ⇒ -1.

```
arr[-1+1] = key
```

arr[0] = 3

0	1	2	3	4	5
3	12	23	45	87	89

⑤
```
for(int i=5; 5<6; i++) {
    int key = arr[5];
    int j = 5-1
    while(4>=0 && 87 > 89){
```

i=5
i=5 ; 5<6 ; 5++
key = 89
j = 4.
4>=0 && 87 > 89 (X)

i=6.

0	1	2	3	4	5
3	12	23	45	87	89

final output.

(iv) Quick sort :

0	1	2	3	4	5
45	23	87	12	3	89.

Given array (arr) = [45, 23, 87, 12, 3, 89]
size (n) = 6.

Calling Methode :

```
static void QuickSortFn(
        int[] arr (in), low, int high){
if(low < high) {
    int pivotIndex = position(arr, low, high);
    QuickSortFn(arr, low, pivotIndex-1); // left part
    QuickSortFn(arr, pivotIndex+1, high); // Right part.
}
}
```

```
static int partition(int[] arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i+1];
    arr[i+1] = arr[high];
    arr[high] = temp;
    return i+1;
}
```

swap

place pivot at correct position

① `QuickSortFn(arr, 0, n-1);`

arr = [45, 23, 87, 12, 3, 89]
low = 0.
high = 6-1 ⇒ 5.
0 < 5 (✓).

```
if (low < high) {
    int pivotIndex = ↓
```

```
partition(arr, 0, 5);
    int pivot = arr[high];
    int i = low - 1;
    for (int j=0; j<5; j++) {
        if (arr[j] < pivot) {
            i++;
            (swap);
```

① inner loop

pivot = 89.
i = 0-1 ⇒ -1.
0 ; 0<5 ; 0++
45 < 89 (✓).
i++ ⇒ 0.
swap: 0 ↔ 0
arr[45, 23, 87, 12, 3, 89]
j = 1.

② outer loop

```
    for (int j=1; j<5; j++) {
        if (arr[j] < pivot) {
            i++
            (swap);
```

j = 1 ; 1<5 ; 1++
23 < 89 (✓).
0++ ⇒ 1

	swap : 1 ↔ 1 arr [45, 23, 87, 12, 3, 89] j = 2.
③ inner loop for (int j = 2 ; j < 5 ; j++) {	j = 2 ; j < 5 ; j++. 87 < 89 (T) ∴ i = 1 + 1 = 2. swap : 2 ↔ 2 arr [45, 23, 87, 12, 3, 89] j = 3.
④ inner loop for (int j = 3 ; j < 5 ; j++) {	j = 3 ; j < 5 ; j++ 12 < 89 (T) ∴ i = 2 + 1 ⇒ 3. swap : 3 ↔ 3 arr [45, 23, 87, 12, 3, 89] j = 4.
⑤ inner loop for (int j = 4 ; j < 5 ; j++) {	j = 4 ; j < 5 ; j++ 3 < 89 (T) ∴ i = 3 + 1 = 4 swap : 4 ↔ 4. arr [45, 23, 87, 12, 3, 89] j = 5.
place pivot correct position.	swap : i = 4 + 1 ⇒ 5 high = 5 arr [45, 23, 87, 12, 3, 89] pivot Index = 5 //
left part. pivot Index = 5. QuickSortFn(arr, 0, pivotIndex - 1) .	low = 0. arr [45, 23, 87, 12, 3, 89] high = 4.
pivotIndex : portion.	

Page No.
Date

① Iteration:
pivot = arr[4] // (3).
int i = low - 1 // (-1).

① Loop iteration.
arr [45, 23, 87, 12, 3, 89]
small
place the pivot.
i = -1 + 1 = 0
swap : 0 ↔ 4
arr [3, 23, 87, 12, 45, 89] //.
partion and pivot Index = 0 //.

Right part:
arr [3, 23, 87, 12, 45, 89] //.
pivotIndex = 0.
low = 0+1 ⇒ 1
high = 4.

partion to get pivotIndex.
pivot = arr[high] = 45/.
i = 1-1 = 0.

① Loop iteration.
i = 1
arr [3, 23, 87, 12, 45, 89]

② Loop iteration (X).
③ Loop iteration
i = 2.
swap [3, 23, 12, 87, 89] (45)

④ place pivot correct position.
swap [3, 23, 12, 45, 87, 89].

Page No.
Date.

Regressive left part.
low = 1
high = 2.
arr [3, 23, 12, 45, 87, 89].
position:
pivot = arr[high] // 12.
① Loop iteration. ✓
② Loop iteration. (x)

place pivot at correct position.
i = 0 + 1
high = 2.
swap : 1 ↔ 2 ⇒ arr [3, 12, 23, 45, 87, 89]

pivot index = 1 //

Final regressive Quick sort result:
arr [3, 12, 23, 45, 87, 89].

b) 90, 34, 12, 67, 22.

i) Bubble sort.
arr [90, 34, 12, 67, 22]
n = 5
Sort:
⇒① outer Loop iteration.
(i = 0 ; 0 < 4 ; i++).
⇒① inter Loop iteration.
(j = 0 ; 0 < 4 - i ; j++).
if 90 > 34 (✓). ⇒ [34, 90, 12, 67, 22]
j = 1

⇒② inner Loop iteration.
(j = 1 ; 1 < 4 - i ; j++)
if 90 > 12 (✓) ⇒ [34, 12, 90, 67, 22]
j = 2.

⇒ ③ Inner Loop Iteration.
(j=2; 2<4; 2++)
if 90 > 67 (✓) ⇒ [34, 12, 67, 90, 22]
J = 3.

⇒ ④ Inner Loop Iteration.
(J=3; 3<4; 3++)
if 90 > 22 (✓). ⇒ [34, 12, 67, 22, 90].
J=4 (X).

⇒ ② Outer Loop Iteration.
(i=1; 1<4; 1++).
⇒ ① Inner Loop Iteration.
(J=0; 0<3; 0++)
if 34 > 12 (✓). ⇒ [12, 34, 67, 22, 90]
J=1

⇒ ② Inner Loop Iteration.
(J=1; 1<3; 1++).
if 34 > 67 (X) J=2

⇒ ③ Inner Loop Iteration.
(J=2; 2<3; 2++) ⇒ [12, 34, 22, 67, 90]
if 67 > 22 (✓) = J=3 (X).

⇒ ③ Outer Loop Iteration.
(i=2; 2<4; 2++).
⇒ ① Inner Loop Iteration.
(J=0; 0<2; 0++)
if 12 > 34 (X). ⇒ J=1.

⇒ ② Inner Loop Iteration
(J=1; 1<2; 1++)
if 34 > 22 (✓). ⇒ [12, 22, 34, 67, 90]
J=2 (X).

=>. ④ outer Loop iteration. i = 3.

=>. ① inner Loop iteration.

(J=0 ; 0 < 1 ; 0++)

if 12 > 22 (X).

0	1	2	3	4.
12	22	34	67	90.

ii). Selection sort.

Given array = [90, 34, 12, 67, 22]

Size = 5.

=> ① outer loop iteration

(i=0 ; 0 < 4 ; 0++). midIndex = 0.

=> ① inner Loop iteration.

(J = 0+1 ; J < 5 ; 1++)

if 34 < 90 (✓). midIndex = 1

J = 2.

=>. ② inner loop iteration.

(J = 2 ; 2 < 5 ; 2++). J = 3.

if 12 < 90 (✓). midIndex = 2.

=> ③ inner loop iteration.

(J = 3 ; 3 < 5 ; 3++). midIndex = 3.

if 67 < 90 (✓). J = 4.

=>. ④ inner loop iteration.

(J = 4 ; 4 < 5 ; 4++) midIndex = 4.

if 22 < 90 (✓). J = 5 (X).

=> Swap : i ↔ midIndex [22, 34, 12, 67, 90]

i = 1.

=> ② outer loop iteration.
(int i=1; i<4; i++)
midIndex = i;

i = 1.
midIndex = 1

=> ① inner loop iteration.
(J = 1+1; 2<5; 2++).
12 < 34 (✓)

midIndex = 2.
J = 3.

=> ② inner loop iteration
(J = 3; 3<5; 3++).
if 67 < 12 (X).

J = 4.

=> ③ inner loop iteration.
(J = 4; 4<5; 4++).
if 90 < 12 (X)

J = 5 (X).

=> Swap: i ⇄ midIndex

[12, 12, 34, 67, 90]
i = 2.

=> ③ outer loop iteration.
(i = 2; 2<4; 2++)
midIndex = i

i = 2.
midIndex = 2.

=> ① inner loop iteration.
(J = i+1; 3<5; 3++)
if 67 < 34 (X).

J = 4.

=> ② inner loop iteration
(J = 4; 4<5; 4++)
if 90 < 34 (X).

J = 5. (X).

=> Swap: i ⇄ midIndex

[12, 22, 34, 67, 90]

0	1	2	3	4
12	22	34	67	90.

iii) Insertion Sort :

Given array : [90, 34, 12, 67, 22]

size : 5

⇒ ① outer loop iteration. i = 1

(i = 1 ; i < 5 ; i++). key = 90.

key = arr[i] j = 1 - 1 ⇒ 0.

j = i - 1 ;

⇒ ① inner loop iteration.

(0 >= 0 && 90 > 90) (X).

arr[0+1] = key [90, 34, 12, 67, 22].

i = 2.

⇒ ② outer loop iteration

(i = 2 ; 2 < 5 ; 2++).

key = arr[2] key = 12

j = 2 - 1 j = 1.

⇒ ① inner loop iteration.

(1 >= 0 && 34 > 12) (✓) [90, 34, 12, 67, 22]

arr[2] = arr[1]

j--

⇒ ② inner loop. iteratio

(0 >= 0 && 90 > 12) (✓) arr[1] = arr[0]

j = -1 (X).

[90, 90, 34, 12, 67, 22]

arr[-1+1] = key. [12, 90, 34, 67, 22].

⇒ ③ outer loop iteration.

(i = 3 ; 3 < 5 ; 3++).

key = arr[3].

j = 3 - 1 ;

① inner loop iteration.

(2 >= 0 && 34 > 22) (✓). [12, 34, 22, 67, 90]

② inner loop iteration (X).

⇒ ④ outer loop iteration.
(i = 4 ; 4 < 5 ; 4++)
key = arr[i].
j = i - 1
⇒ ① inner loop iteration. (X)
arr[j+1] = key.

⇒

0	1	2	3	4
12	22	34	67	90

iv). Quick sort :
Given array : [90, 34, 12, 67, 22]
size : 5.

⇒ Initial Quick sort.
low = 0.
high = n-1 ⇒ 4.

⇒ pivot Index :
pivot Element : arr[high] 22 //.
i = low - 1 ⇒ -1
⇒ ① Loop iteration.
(j = 0 ; 0 < 4 ; j++).
If 90 < 22 (X).
⇒ ② Loop iteration.
(j = 1 ; 1 < 4 ; j++)
If 34 < 22 (X).
⇒ ③ Loop iteration.
(j = 2 ; 2 < 4 ; j++). i++ ⇒ 0.
If 12 < 22 (✓). Swap i ⇄ j ⇒ 0 ⇄ 2
[12, 34, 90, 67, 22].

⇒ ④ Loop iteration.
(j = 3 ; 3 < 4 ; j++).
If 67 < 22 (X).

Page No.
Date

=> place pivot correct position.

(i+1) ⇄ high => ① ⇄ 4

[12, 22, 90, 67, 34].

pivotIndex = 1 //

=> Left part Quick Sort.

low = 0 (X).

high = pivotIndex - 1 => 0.

low < high (X).

=> Righ part Quick Sort.

low = pivotIndex + 1 => 2.

high = 4.

=> pivotIndex :

pivot = arr [high] . 34 //.

i = 2 - 1 1 //

① loop iteration.

(i = 2 ; 2 < 4 ; 2++).

if 90 < 34 (X).

② loop iteration.

(i = 3 ; 3 < 4 ; 3++).

if 67 < 34 (X).

Place pivot correct position.

(i+1) ⇄ high => 2 ⇄ 4

[12, 22, 34, 67, 90].

0	1	2	3	4
12	22	34	67	90